

자체학습 Qwen vs Claude API — 정량·품질 비교

동일 케이스 23건 · 동일 채점기(judge.py) · 09-11 / 09-12 / 09-17 실측. 모든 수치는 저장소 eval·벤치 데이터 인용 · 작성 2026-09-22

한 줄 결론 — 같은 자로 재니 학습 전 Qwen **26.1%** → Claude **69.6%** 로 격차가 컸고, 자체학습 v9가 **73.9%**로 넘어섰다. 다만 **품질의 성격**이 다르다: Qwen(학습 전)은 "판단은 맞는데 도구 언어를 못 말함", Claude는 "안전한 순서로 가다 단일턴 채점에 걸림", v9는 "남은 실패 6건 중 5건이 UX상 정상". 심사 서빙은 품질·안정성·트래픽 이유로 클라우드+Claude 확정.

1 도구 호출 정확도 — 같은 자(judge.py) · 23건

Qwen 학습 전 · 09-11

26.1 % · 6/23

도메인 미스매치 — 도구 포맷 미학습

Claude Haiku 4.5 · 09-12

69.6 % · 16/23

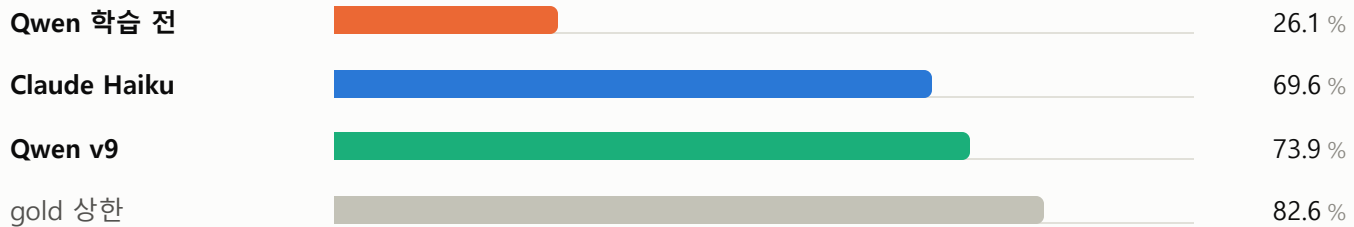
gold 상한 19/23 대비 84%

Qwen v9 학습 후 · 09-17

73.9 % · 17/23

v6-v7-v8 상회 · 실질 ~95%+

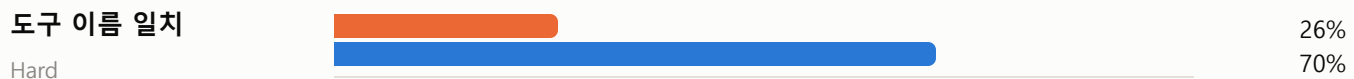
pass = Hard 3개(도구 이름·no-tool 위반 없음·확인 문구) 전부 + Soft(인자 또는 키워드) 하나 이상. 이론상 최대 (gold 상한) 19/23.



2 품질 분해 ① — 판정 항목별 통과율

"맞았다/틀렸다"를 5개 항목으로 쪼개면 어디서 무너졌는지가 보인다. (v9는 항목별 원자료 미보존 → 총점만)

Qwen 학습 전 Claude Haiku



no-tool 위반 없음

Hard · 부르지 말아야 할 때 안
부름



100%
96%

확인 문구

Hard · 쓰기 도구 전 "진행할까
요?"



91%
87%

인자·응답 문구

Soft · 둘 중 하나



78%
83%

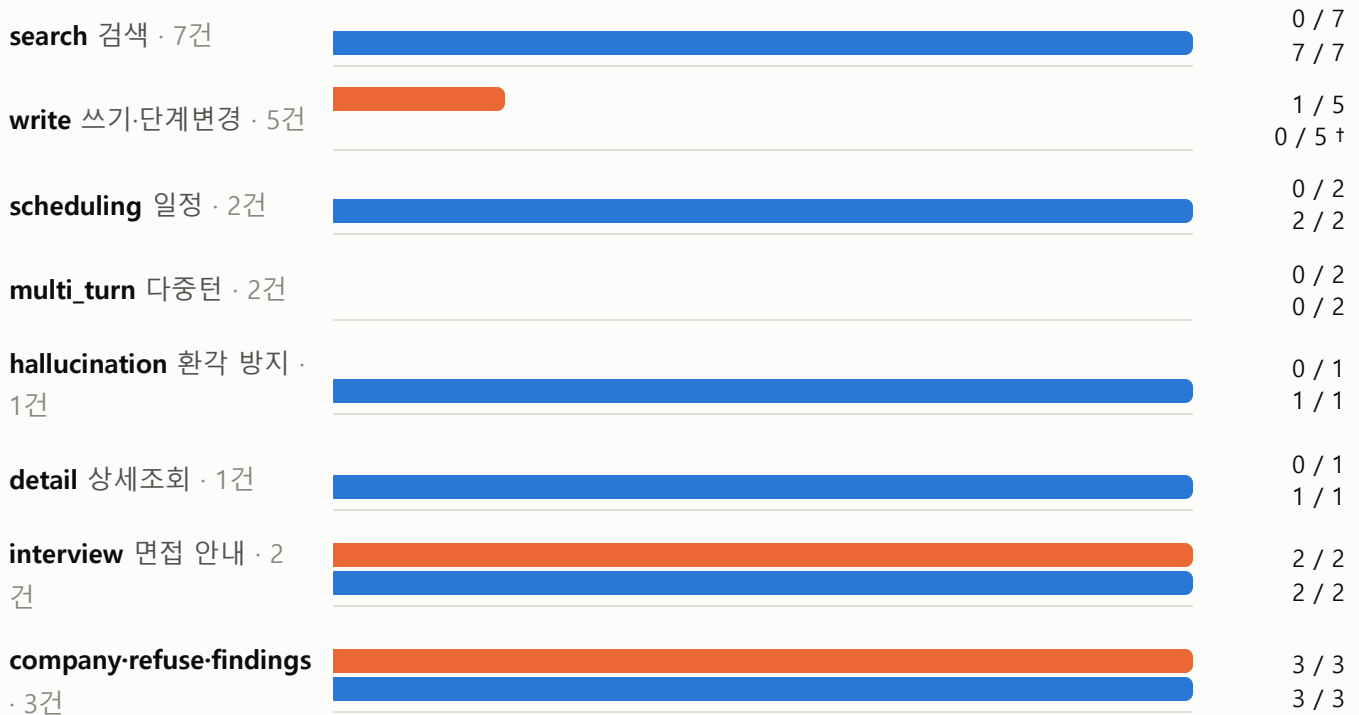
읽는 법: 학습 전 Qwen의 약점은 딱 하나 — 도구 이름 일치(26%). 나머지 셋(위반 없음 100%·확인 문구 91%·응답 78%)은 Claude와 비슷하거나 낫다. 즉 "무엇을 해야 하는지"는 알았는데 `<tool_call>` 포맷을 못 냈다 — `<think>` 안에서 "search_applications 를 불러야 한다"고 추론하고도 실제 호출을 안 한 케이스가 대부분. 이것이 도메인 미스매치의 실체이고, QLoRA 학습이 정확히 이 한 축을 고쳐 v9 73.9%가 됐다.

▶ 표로 보기

3 품질 분해 ② — 질문 유형별 정확도

유형마다 강약이 갈린다. 막대는 통과 건수 / 유형 건수.

■ Qwen 학습 전 ■ Claude Haiku



† Claude write 0/5 는 단일턴 채점의 한계. Claude는 대상을 먼저 `search` 로 찾고 확인 턴을 기다리는 안전한 순서를 택해, 한 턴 안에 `change_stage` 를 내지 않아 미완료로 잡혔다. 실사용(다중턴)에선 정상 동작이고, 오히려 "부탁 안 한 쓰기 도구를 지어내지 않았다"는 안전 신호다.

반대로 학습 전 Qwen은 **거절·경계(company-refuse)**와 **면접 안내는 만점** — 채용 밖 질문을 거절하고 개인정보를 안 내는 판단력은 처음부터 있었다.

4 품질 분해 ③ — 실패의 "질" (안전한 실패 vs 진짜 실수)

틀린 건수보다 **어떻게 틀렸나**가 운영 리스크를 결정한다. 상태 색은 아이콘·라벨과 함께 쓴다.

Qwen v9 · 실패 6건 (17/23 통과)



✓ UX상 정상 5건 — 확인 문구 통과·안전한 다단계 진행(채점기만 미완료로 봄)

✗ 진짜 실수 1건 — `list_postings` 오선택

→ 실질 정확도 ~95%+. 지어낸 쓰기 도구(hallucinated write) **0건**.

Claude Haiku · 실패 7건 (16/23 통과)



✓ 안전한 순서 5건 — 대상 검색 후 확인 대기(write 유형)

! 다중턴 맥락 미반영 2건 — 앞 턴의 대상을 이어받지 못함

→ 위험한 실패(임의 실행·지어낸 도구) **0건**. no-tool 위반 1건은 "확인만 요청한 상황에 검색을 한 번 더 한" 과잉 조회.

Qwen 학습 전 · 실패 17건 (6/23 통과)



! 도구 미호출 17건 — 추론은 맞으나 `<tool_call>` 포맷을 내지 못함 (포맷 실패, 판단 실패 아님)

→ 임의 실행·환각 도구 **0건**. 실패가 전부 "안 함"이라 데이터 훼손 위험은 없었다 — 다만 쓸 수 없는 수준.

5 품질 분해 ④ — 서류 요약 어댑터 (정성 품질)

도구 호출과 별개로, 서류 요약 전용 어댑터(`qwen3-arda-summary`)의 출력 품질을 항목별로 봤다.

항목	Qwen summary 어댑터	Claude Haiku (v2 체인)
JSON 형식 준수	✓ 완벽 (파싱 실패 0)	✓ (파싱 폴백 있음)
gist(요지) 정확성	✓ 정확	✓ 정확

key_skills 추출	✓ 정확	✓ 정확
career_years(경력 연수)	✗ 오차 — 정답 3년을 5년으로 (인턴 기간 포함)	✓ (v2 스키마로 채움)
예시 숫자 베끼기	✗ 소형모델 특성 — 프롬프트 예시 70/40/60 을 그대로 냄 (→ v3 프롬프트로 예시 제거해 해결)	✓ 안 베낌
문자열 내 따옴표 깨짐	✗ 발생 (→ 파서 복구 폴백 추가로 56/56 성공)	✓ 없음

해석: 요약 어댑터는 "형식·요지·역량"은 Claude 수준이지만, 수치 추론(경력 연수)·프롬프트 예시 분리에서 소형모델 한계가 드러났다. 이 두 결함은 프롬프트(v3)·파서 폴백으로 우회했지만, 매번 땀질이 필요한 성격이라 심사 서빙을 Claude로 확정한 근거 중 하나다.

6 비용

Claude · eval 1건당

\$0.0139

23건 \$0.3199 · in 307,497 / out 2,480 tok

Claude · 실사용 1호출당

\$0.0075

아르 로그 23건 누적 \$0.1714

Qwen · 1호출당

\$0 +GPU

GPU 상시 \$0.71/h · 손익분기 ≈ 하루 51면접

항목	Claude API	Qwen 자체 서빙
과금 구조	사용량(토큰)만 · 고정비 0	호출당 0 · GPU 고정비 \$0.71/h(+EBS \$0.014/h)
기본·경계 질문	캔드 응답 \$0 (FAQ)	—
STT	OpenAI whisper-1 \$0.006/min ≈ \$0.06/면접 (클라우드 서빙 기본 경로)	

※ eval 1건(\$0.0139)이 실사용(\$0.0075)보다 비싼 이유: eval은 전체 시스템 프롬프트를 매 건 포함(입력 13.4K tok/건). 실사용은 캐시·짧은 대화로 더 싸다.

7 속도 · 지연 (자체 서빙 실측)

구성	지표	실측
----	----	----

Qwen chat (Ollama Q4_K_M · T4)	생성 속도 · TTFT · 콜드 스타트	39.5 tok/s · ~40ms · 78초
Qwen 10회 벤치	92 tok → 1,024 tok	2.3s → 26.4s
STT (GPU · faster-whisper large-v3-turbo fp16)	3초 음성 전사	275ms
표정 ViT (GPU fp16 · VRAM 178MB)	crop 1장	75ms
Claude API	응답 지연	이 데이터셋엔 미측정

운영 관점: Qwen의 콜드 스타트 78초와 긴 응답(1K tok = 26초)은 공채 시즌 동시 접속에서 대기줄을 만든다 (부하테스트: 8명 동시 시 STT 대기 15→42초). Claude는 오토스케일로 이 부류 문제가 없다 — 정확도 격차보다 이 안정성 차이가 클라우드 확정의 더 큰 이유다.

8 음성 전사(STT) 품질 이슈 — 둘 다 겪은 것

이슈	OpenAI whisper-1 (API)	faster-whisper (로컬)
무음·잡음에 말 지어냄 (hallucination)	× 발생 — 유튜브 자막 문구가 답변으로 저장된 사고(09-15)	× 발생 — 무음 3초가 "감사합니다."로
완화 조치	✓ <code>no_speech_prob</code> ≥ 0.6 조각 폐기	✓ VAD 필터 + 이전 문맥 미참조
실시간 예산(1.5초) 준수	✓ API 응답	× CPU(t3.large)에서 44초 발화 → 180초 초과 · 연쇄 지연

전사 정확도 자체는 두 엔진 비슷(재현 클라이언트 12/12 정확). 차이는 지연과 무음 처리였고, 클라우드는 API + 게이트로 정리했다.

9 해석 주의 — 숫자를 과신하지 않기 위해

- **채점기 편향:** 테스트셋· `judge.py` 는 원래 Qwen 학습(gold) 기준으로 만들어져 약간 Qwen 친화적이다. 절대 점수보다 같은 자·같은 편향으로 낸 상대 꺾적(26.1 → 69.6 → 73.9)이 유효한 지표다.
- **표본 크기:** 23건은 방향을 보기엔 충분하지만 1~2건 차이(예: 69.6 vs 73.9)는 통계적으로 유의하다고 말하기 어렵다. "v9가 Claude와 동급"이 정직한 표현이다.
- **단일턴 채점:** 다중턴 확인 흐름(검색→확인→실행)을 한 턴에 평가해 Claude write 5건이 저평가됐다.
- **v9 항목별 원자료**는 보존되지 않아 총점과 실패 분류만 있다.

정리 — 정확도는 v9가 Claude와 동급(73.9 vs 69.6)까지 올라왔고, 실패의 질도 둘 다 "위험한 실패 0건"으로 안전하다. 갈린 곳은 **수치 추론·형식 안정성(요약 어댑터)**과 **지연·동시성(콜드 78초·대기줄)**이며, 이것이 심사 서빙을 클라우드+Claude로 확정한 이유다. Qwen 자체학습은 "public API 없이도 갈 수 있는가"에 실측으로 답한 **비용 절감 R&D 자산**으로 남는다.

Team Seuk · Arda — 2026-09-22 · 출처: `ai/qwen-training` (judge-eval) · 09-11 로컬 Qwen eval · 09-12 Claude Haiku eval(`eval_anthropic.py`) · 09-17 v9·GPU 벤치 · 비용비교 데이터(JSON 4세그먼트). 팔레트: 검증된 기본 카테고리 3슬롯(파랑·주황·아쿠아, 전 쌍 CVD $\Delta E \geq 9$) · 상태색은 아이콘+라벨 동반.